

Modifica dei dati

DELETE

Come eliminare righe da una tabella senza cancellare più del necessario.

Cancellare righe da una tabella

`DELETE` serve a rimuovere righe da una tabella.

Non elimina la tabella intera. Elimina solo i dati che scegli con il filtro.

Per esempio:

```
DELETE FROM clienti  
WHERE id = 3;
```

Questa query cancella il cliente con `id = 3`.

Il filtro è la parte più importante

Con `DELETE`, `WHERE` è la tua cintura di sicurezza.

```
WHERE id = 3
```

Senza questa condizione, il database potrebbe cancellare tutte le righe della tabella.

Attenzione: Prima di eseguire un `DELETE`, controlla sempre quali righe stai per cancellare.

Il controllo con **SELECT**

Il modo più prudente è provare prima la stessa condizione con una **SELECT** :

```
SELECT *  
FROM clienti  
WHERE id = 3;
```

Se il risultato è proprio la riga che vuoi eliminare, puoi usare lo stesso filtro nel **DELETE** .

Altro esempio:

```
SELECT *  
FROM clienti  
WHERE email = 'test@email.it';
```

Se vedi solo il cliente di test, puoi cancellarlo così:

```
DELETE FROM clienti  
WHERE email = 'test@email.it';
```

La condizione è identica. Cambia solo l'azione.

DELETE non è **DROP TABLE**

È facile confonderli all'inizio, ma fanno cose molto diverse:

- **DELETE** elimina righe.
- **DROP TABLE** elimina la tabella intera, compresa la struttura.

Se immagini una rubrica, **DELETE** cancella uno o più contatti. **DROP TABLE** butta via tutta la rubrica.

A volte è meglio non cancellare

Nel mondo reale, non sempre conviene eliminare davvero una riga.

Per esempio, se un cliente ha fatto ordini o ricevuto fatture, potresti voler conservare la sua storia. In questi casi spesso si aggiunge una colonna come `attivo`.

```
UPDATE clienti
SET attivo = FALSE
WHERE id = 3;
```

Così il cliente non risulta più attivo, ma i suoi dati restano disponibili per controlli futuri.

Riepilogo rapido

- `DELETE` elimina righe.
- `WHERE` decide quali righe eliminare.
- Prima di cancellare, controlla il filtro con una `SELECT`.
- `DELETE` non elimina la struttura della tabella.
- In alcuni casi conviene disattivare una riga invece di cancellarla.

INSERT

Come aggiungere nuove righe a una tabella in modo chiaro e sicuro.

Aggiungere una nuova riga

`INSERT` serve quando vuoi **mettere nuovi dati dentro una tabella**.

Se la tabella è una rubrica, `INSERT` aggiunge un nuovo contatto. Se è un catalogo prodotti, aggiunge un nuovo prodotto.

Ecco un esempio semplice:

```
INSERT INTO clienti (nome, email)
VALUES ('Luca', 'luca@email.it');
```

Puoi leggerlo così:

"Nella tabella `clienti`, riempi le colonne `nome` ed `email` con questi valori."

Il primo valore, `'Luca'`, finisce nella prima colonna indicata, cioè `nome`. Il secondo valore, `'luca@email.it'`, finisce in `email`.

Perché scrivere i nomi delle colonne

SQL permette anche forme più brevi, ma quando stai imparando è meglio indicare sempre le colonne.

Questa forma è poco chiara:

```
INSERT INTO clienti
VALUES (1, 'Luca', 'luca@email.it');
```

Funziona solo se conosci esattamente l'ordine delle colonne nella tabella.

Questa invece si legge meglio:

```
INSERT INTO clienti (id, nome, email)
VALUES (1, 'Luca', 'luca@email.it');
```

Anche tra qualche mese capirai subito cosa stai inserendo e dove.

Inserire più righe insieme

Se devi aggiungere più clienti, puoi farlo con un solo `INSERT`:

```
INSERT INTO clienti (nome, email)
VALUES
  ('Marta', 'marta@email.it'),
  ('Paolo', 'paolo@email.it');
```

Ogni coppia tra parentesi crea una nuova riga.

Quando manca un valore

Non tutte le colonne devono essere sempre compilate.

Se una colonna è facoltativa, puoi non inserirla. Se invece è obbligatoria, per esempio perché ha **NOT NULL**, il database rifiuterà la riga.

Per esempio, se **nome** è obbligatorio, questa query può fallire:

```
INSERT INTO clienti (email)
VALUES ('senza.nome@email.it');
```

Il database ti sta proteggendo da un dato incompleto: non vuole salvare un cliente senza nome.

Attenzione ai tipi di dato

Ogni colonna si aspetta un certo tipo di valore.

```
INSERT INTO prodotti (nome, prezzo, disponibile)
VALUES ('Tastiera', 39.90, TRUE);
```

Qui stai inserendo:

- un testo: **'Tastiera'**
- un numero decimale: **39.90**
- un valore vero/falso: **TRUE**

Non tutti i database scrivono i valori booleani nello stesso identico modo, ma l'idea resta la stessa: ogni valore deve essere adatto alla sua colonna.

Prima di eseguire un **INSERT**

Fai un controllo veloce:

- sto compilando le colonne obbligatorie?
- i valori sono nell'ordine giusto?
- ogni valore ha il tipo corretto?

Sono domande semplici, ma evitano molti errori.

Riepilogo rapido

- `INSERT INTO` aggiunge nuove righe.
 - Conviene indicare sempre le colonne.
 - `VALUES` contiene i dati da inserire.
 - L'ordine dei valori deve seguire l'ordine delle colonne.
 - Il database può rifiutare dati mancanti o del tipo sbagliato.
-

UPDATE

Come modificare dati già presenti nel database senza toccare le righe sbagliate.

Modificare una riga esistente

`UPDATE` serve quando un dato esiste già, ma deve cambiare.

Un cliente cambia città. Un prodotto cambia prezzo. Un ordine passa da "in preparazione" a "spedito". In tutti questi casi non vuoi creare una nuova riga: vuoi aggiornare quella giusta.

Ecco la forma base:

```
UPDATE clienti
SET citta = 'Bologna'
WHERE id = 1;
```

Questa query dice:

"Nella tabella `clienti`, cambia la città in `Bologna`, ma solo per la riga con `id = 1`."

Il ruolo di `SET`

`SET` indica cosa deve cambiare:

```
SET citta = 'Bologna'
```

Puoi aggiornare anche più colonne insieme:

```
UPDATE clienti  
SET citta = 'Bologna', email = 'luca.nuova@email.it'  
WHERE id = 1;
```

Qui cambi sia la città sia l'email dello stesso cliente.

Il ruolo di WHERE

La parte più delicata è WHERE .

```
WHERE id = 1
```

Questa condizione sceglie la riga da modificare. Senza di lei, il database non sa che vuoi aggiornare un solo cliente.

Attenzione: Un `UPDATE` senza `WHERE` può modificare tutte le righe della tabella.

Una buona abitudine

Prima di fare un aggiornamento importante, prova la condizione con una `SELECT` :

```
SELECT *  
FROM clienti  
WHERE id = 1;
```

Se la `SELECT` mostra una sola riga, sai che l' `UPDATE` toccherà una sola riga. Se ne mostra molte, fermati e controlla meglio il filtro.

Usare il valore attuale

A volte il nuovo valore dipende da quello già presente.

Per esempio, questa query aumenta del 10% il prezzo di un prodotto:

```
UPDATE prodotti
SET prezzo = prezzo * 1.10
WHERE id = 5;
```

`prezzo * 1.10` significa: "prendi il prezzo attuale e moltiplicalo per 1,10".

È una possibilità utile, ma va usata con attenzione. Anche qui, il filtro `WHERE` decide quali righe cambieranno.

Riepilogo rapido

- `UPDATE` modifica righe già esistenti.
- `SET` dice quali colonne cambiare.
- `WHERE` dice quali righe modificare.
- Prima di un aggiornamento importante, controlla il filtro con una `SELECT`.
- Senza `WHERE`, rischi di aggiornare tutta la tabella.